Optimized Local Ingestion and Thematic Motif Extraction Utilizing 768-Dimensional Embedding Architectures

The automated extraction of thematic motifs from unstructured document collections is a highly complex task in natural language processing[1]. Unlike standard information retrieval systems designed to locate specific keywords or isolated facts, motif extraction aims to identify recurring conceptual structures, narrative patterns, or thematic sequences that span across a corpus[1]. These motifs are often expressed through highly diverse vocabulary and varied syntactic structures, rendering basic lexical search techniques ineffective[1].

To extract these motifs locally on commodity hardware, developers require highly optimized embedding models, layout-aware document parsers, and sophisticated clustering architectures[7]. This report evaluates the performance of local embedding models centered around the 768-dimension tier, details layout-aware ingestion pipelines for PDF files, examines advanced text segmentation strategies, and demonstrates how to assemble and optimize an end-to-end thematic motif extraction pipeline.

Comparative Analysis of Local 768-Dimensional Embedding Models

Embedding models operating at the 768-dimension level represent a critical efficiency threshold for local enterprise deployment[10]. These models are typically built on optimized Bidirectional Encoder Representations from Transformers (BERT) backbones, enabling them to capture deep contextual relationships while maintaining a compact memory footprint[7]. When extracting thematic motifs, the ideal embedding model must balance retrieval accuracy, context window length, and support for dimensionality truncation via Matryoshka Representation Learning (MRL)[7].

The transition from legacy embedding models to modern 768-dimensional local architectures is driven by several critical design changes. Models such as nomic-embed-text-v1.5 replace absolute positional encodings with Rotary Position Embeddings (RoPE), integrate SwiGLU activation functions instead of GeLU, incorporate Flash Attention mechanisms, and utilize zero-dropout training[12]. These modifications allow the model to handle long-context sequences (up to 8,192 tokens) on standard consumer hardware without experiencing severe memory degradation or sequence length limitations[7]. This stands in contrast to standard BERT architectures which are traditionally capped at 512 tokens[7].

Matryoshka Representation Learning allows a single embedding model to output resizable vectors without requiring multiple retraining passes[7]. This is accomplished by structuring the

loss function during contrastive training such that the early dimensions of the vector (e.g., the first 256 or 128 indices) contain the highest-density semantic weights[10]. Truncating a 768-dimensional embedding to 256 dimensions using snowflake-arctic-embed-m-v1.5 yields a 12x reduction in index size and storage requirements when combined with scalar quantization[10]. This occurs with a marginal performance degradation of less than 2% on retrieval tasks[10]:

$$\text{Truncation Loss Ratio} = \frac{\text{NDCG@10}_{\text{256-d}}}{\text{NDCG@10}_{\text{768-d}}} = \frac{54.2}{55.1} \approx 98.4\% \text{ Performance Retention [cite: 10, 17, 18]}$$

Uniform scalar quantization maps continuous floating-point values to discrete low-bit representations[10]:

$$x_{\text{quant}} = \text{clip}\left(\text{round}\left(\frac{x}{\Delta}\right), -2^{b-1}, 2^{b-1}-1\right) \text{ [cite: 10, 17]}$$

Where b represents the target bits (e.g., $b = 8$ or $b = 4$), and Δ is the quantization step derived from optimal scalar boundaries, such as the $[-0.3, 0.3]$ range for 8-bit configurations and $[-0.18, 0.18]$ for 4-bit configurations[10].

Bi-encoder architectures inherently suffer from semantic symmetry, mapping queries and documents into the same spatial orientation regardless of structural variations[13]. To improve thematic clustering, models like nomic-embed-text-v1.5 integrate task-specific instruction prefixes[13]. Applying the clustering prefix ensures that the spatial distribution of vectors is optimized for density-based clustering rather than pairwise similarity matching, leading to highly defined semantic groupings[19].

The following table compares the leading open-weight embedding models optimized for local deployment in the 768-dimension range:

Model Identifier	Parameter Count	Maximum Context Window	Native Dimensions	Matryoshka Learning Support	License Type	MTEB Retrieval Average (NDCG@10)
nomic-embed-text-v1.5	137 Million[7]	8,192 Tokens[7]	768[19]	Yes (Truncates to 512, 256, 128, 64)[7]	Apache 2.0[14]	53.25[20] / 62.28 (Avg)[12]

bge-base-en-v1.5	109 Million[22]	512 Tokens[16]	768[22]	No[23]	MIT[22]	53.25[20] / 63.55 (Avg)[23]
snowflake-arctic-embed-m-v1.5	109 Million[10]	512 (8,192 via RPE)[20]	768[21]	Yes (Truncates to 256, 128)[10]	Apache 2.0[10]	55.14[17] / 54.20 (at 256-d)[10]
gte-base-en-v1.5	110 Million[24]	512 Tokens[23]	768[23]	No[23]	Apache 2.0[25]	51.14[23] / 62.39 (Avg)[23]
nomic-embed-text-v2-moe	475M (305M active)[14]	512 Tokens[14]	768[14]	Yes (Truncates to 256)[14]	Apache 2.0[26]	~100 Languages supported[14]

High-Fidelity PDF Ingestion and Structural Layout Extraction

Extracting coherent themes from PDF documents is heavily dependent on the layout fidelity of the raw text extraction phase[8]. Traditional programmatic parsers (e.g., PyMuPDF) extract text based on drawing coordinates, which strips visual formatting, breaks reading order in multi-column research papers, and mixes headers, footers, and table cells into a chaotic text sequence[8]. This sequence fragmentation disrupts the contiguous semantic flows necessary for context-aware embedding models to function correctly[30].

To mitigate context fragmentation, layout-aware document parsers must be utilized[8]. Docling employs a multi-stage linear pipeline optimized for semantic parsing[33]. The underlying layout object detector is an RT-DETR model trained on canonical-DocLayNet, a dataset of 150,000 single-page documents containing over 2.3 million structural elements[33]. The pipeline filters out the noisy "delta" classes (e.g., checkboxes, forms, index blocks) using low confidence thresholds (typically 0.3) to ensure clean structural boundaries during inference[34]. It classifies visual zones into 11 distinct labels: Caption, Footnote, Formula, List-item, Page-footer, Page-header, Picture, Section-header, Table, Text, and Title[35].

Tables detected by the layout model are routed to TableFormer, a specialized vision-transformer model[33]. TableFormer predicts both row/column grid lines and multi-level span structures directly from the table's visual bounding box[33]. This dual-model approach allows Docling to preserve table contexts, such as nested structures and lists within cells, which

are formatted into structured Markdown or JSON representation[8].
The following table evaluates the performance of leading programmatic and layout-aware document parsers on diverse PDF layouts:

Parser System	Core Ingestion Architecture	Execution Speed (Pages/Sec)	Table Cell Extraction Accuracy	Primary Limitations & Failure Modes
Docling (IBM)	RT-DETR Layout Object Detection + TableFormer[28]	3.2 pages/sec[28]	97.9%[28]	High computational overhead on single-core CPUs; occasional cell misalignment in extremely dense borderless matrices[28].
Marker (Datalab)	Surya Layout Engine + Vision Transformer Pipeline[28]	6.1 pages/sec[28]	91.7%[28]	Struggles with multi-level headers and borderless tables using visual whitespace alignment[28].
Unstructured.io	Detectron2 Layout Detection + Optional OCR[28]	4.8 pages/sec (Hi-Res)[28]	Variable (~75% on complex tables)[37]	Significant column-shift errors in multi-page hierarchical tables; slow ingestion speed under high-resolution processing[37].
PyMuPDF (fitz)	Coordinate-Based Direct Text	>12 pages/sec	0.0% (Text is	Total loss of structural

	Extraction[29]		flattened)	context in multi-column PDFs, tables, or scanned documents; no native OCR[29].

Structural, Semantic, and Cross-Document Chunking Paradigms

Once structured Markdown is extracted, it must be segmented into text chunks optimized for the downstream embedding model[30]. Standard recursive character splitters divide text at static character thresholds, frequently splitting paragraphs mid-concept and decoupling critical sub-arguments from their parent headings[31]. To extract motifs with high precision, the segmentation strategy must adapt to both the semantic shifts in the prose and the document's physical layout[42].

Algorithmic Implementation of Semantic Chunking

Semantic chunking uses embedding distance to identify topic transitions[41]. The mathematical implementation of this algorithm consists of six consecutive steps:

1. **Sentence Splitting**: The raw document text is tokenized into individual sentences:

$$S = \{s_1, s_2, \ldots, s_N\} \text{ [cite: 41, 45]}$$

1. **Vectorization**: Each sentence is embedded using a high-fidelity local model (such as all-MiniLM-L6-v2 or all-mpnet-base-v1):

$$\mathbf{e}_i = \text{Embed}(s_i) \in \mathbb{R}^D \text{ [cite: 41, 45]}$$

2. **Window-Based Contextualization**: To ensure semantic continuity, each sentence vector is combined with adjacent sentences within a fixed sliding window (w):

$$\mathbf{c}_i = \frac{1}{2w+1} \sum_{j=i-w}^{i+w} \mathbf{e}_j \text{ [cite: 45]}$$

3. **Cosine Distance Computation**: The system calculates the cosine distance between the contextualized vectors of consecutive sentences[42]:

$$d_i = 1 - \frac{\mathbf{c}_i \cdot \mathbf{c}_{i+1}}{\|\mathbf{c}_i\|_2 \|\mathbf{c}_{i+1}\|_2} \text{ [cite: 42, 45]}$$

4. **Threshold Determination**: Breakpoints are identified when the distance exceeds a dynamic threshold (T). The threshold is calculated using a percentile-based or standard deviation metric of the total distance distribution across the document:

$$T = \mu_d + k \cdot \sigma_d \text{ [cite: 43, 45]}$$

Where μ_d is the mean cosine distance, σ_d is the standard deviation, and k is an adjustable scaling parameter (typically 1.2 to 1.5)[43].

6. **Chunk Construction**: Sentences are grouped into a continuous chunk until a boundary where $d_i > T$ is encountered, forcing the creation of a new semantic block[41].

Layout-Aware and Hybrid Chunking

While purely embedding-based semantic chunking excels at identifying transitions in unstructured narrative prose, it can perform poorly when applied to highly structured technical documents or multi-page reports[42]. Highly technical text often repeats domain-specific vocabulary across sections, generating a false high-similarity signal that prevents the embedding model from identifying structural transitions[47].

To solve this, Docling's HybridChunker combines tokenization-aware limits with the hierarchical tree parsed during ingestion[40]. The hybrid chunking protocol operates under three structural principles[40]:

- **Hierarchy Preservation**: Chunks are initially bounded by the document's section headings[40]. If a section is small, it remains a single chunk. If it exceeds the target token limit, it is split recursively at structural markers (e.g., list-item boundaries, table rows, or paragraph breaks) rather than arbitrary sentence counts[40].
- **Context Prepending**: When a section is subdivided, the titles of all parent sections (e.g., # Introduction $\rightarrow$ ## Core Architecture) are prepended to the text of each sub-chunk[49]. This preserves the logical position of the text, preventing context loss during retrieval[31].
- **Table Header Repetition**: For tabular structures that exceed the token limit, the chunker splits the table along row boundaries and repeats the table headers at the beginning of each subsequent chunk[40]. This ensures that cell values isolated on later pages retain their semantic association with their respective columns[40].

Advanced Core-Level Context Preservation

For long documents, contextual dependencies can be preserved at the vector level using Late Chunking or Cross-Document Topic-Aligned (CDTA) chunking[31].

Late chunking first processes the entire un-split document through the transformer attention layer, allowing the token embeddings to capture the context of the surrounding paragraphs[31]. The document is then split into smaller segments, and chunk-level vectors are derived using mean pooling over the pre-computed token matrices[31]. This approach preserves long-distance dependencies within the document[31].
For corpus-level thematic analysis, CDTA chunking bypasses individual document boundaries to reconstruct global themes[31]. It maps segments across the entire collection to a set of globally discovered topics and synthesizes them into cohesive, cross-document chunks[31]. On the QuALITY and HotpotQA multi-hop reasoning tasks, CDTA chunking achieves a 0.93 faithfulness score compared to 0.83 for contextual retrieval and 0.78 for isolated semantic chunking, maintaining a 0.91 faithfulness score even at a low retrieval count of $k = 3$ [31].

Advanced Thematic Clustering Pipelines and Descriptors

Following document chunking, thematic motifs are extracted using an unsupervised, modular topic-modeling framework[9]. The classical pipeline consists of four sequential stages: embedding generation, non-linear dimensionality reduction, density-based clustering, and class-based representation[9].

High-Dimensional Projection via UMAP

High-dimensional semantic spaces (such as the 768-dimensional space generated by local models) suffer from the "curse of dimensionality"[6]. In these spaces, the distance between any two vectors converges, making distance-based clustering algorithms ineffective[6]. To resolve this, Uniform Manifold Approximation and Projection (UMAP) is used to project the embeddings into a lower-dimensional space (typically 5 to 10 dimensions)[6].
UMAP models the high-dimensional data using fuzzy simplicial sets and optimizes the low-dimensional representation to preserve both local and global topological structures[9]. The algorithm is controlled by two key hyperparameters[6]:

- n_neighbors: Bounds the size of the local neighborhood used to construct the manifold[6]. Low values (e.g., 10 to 15) prioritize the preservation of local structure, which is ideal for identifying highly specific micro-motifs[50]. Higher values (e.g., 50 to 100) prioritize global structure, grouping broad themes[51].
- n_components: Specifies the target dimensionality[6]. For downstream density clustering, a range of $5 \leq \text{dimensions} \leq 10$ is recommended to balance information loss with distance metrics[6].

Density-Based Partitioning via HDBSCAN

To identify motifs of varying density and spatial geometry without defining the number of clusters in advance, Hierarchical Density-Based Spatial Clustering of Applications with Noise

(HDBSCAN) is applied to the UMAP coordinates[6]. HDBSCAN defines cluster centroids based on spatial density rather than radial distance, converting DBSCAN into a hierarchical clustering framework[9]. The process relies on mutual reachability distance to calculate density, defined for two points $\mathbf{x}_i$ and $\mathbf{x}_j$ as[9]:

$$d_{\text{mrc}}(\mathbf{x}_i, \mathbf{x}_j) = \max\{\text{core}_k(\mathbf{x}_i), \text{core}_k(\mathbf{x}_j), d(\mathbf{x}_i, \mathbf{x}_j)\} \text{ [cite: 9]}$$

where $d(\mathbf{x}_i, \mathbf{x}_j)$ is the Euclidean distance and $\text{core}_k(\mathbf{x})$ is the distance to its k-th nearest neighbor[9]. This density metric allows HDBSCAN to isolate dense clusters from surrounding low-density noise, which are assigned to a noise category (labeled as -1)[6]. The granularity of extracted motifs is controlled by the min_cluster_size parameter, which dictates the minimum number of chunks required to establish a distinct motif[6].

Pipeline Optimization via Modular Enhancements

Standard implementations of density-based clustering are often limited by noise sensitivity, static configurations, and high-frequency terms in technical text[9]. To address these challenges, three modular enhancements are integrated into the pipeline:

- **Popularity Deviation Regularizer (PDR)**: Highly technical documents often contain repetitive, domain-general terms (e.g., "system," "data," "process") that can dominate c-TF-IDF keyword calculations[9]. The PDR applies an exponential decay penalty to these high-frequency generic words based on their global frequency rank, while upweighting domain-specific terminology to improve topic distinctiveness[9]:

$$W_{\text{pdr}}(t) = e^{-\alpha \cdot \text{rank}(t)} \times \text{Multiplier}_{\text{domain}}(t) \text{ [cite: 9]}$$

- **Dynamic Document Embedding Optimizer (DDEO)**: To prevent information loss during dimensionality reduction, the DDEO dynamically optimizes UMAP's projection dimensions[9]. Rather than using a static component count, the system sweeps through target dimensions ($3 \leq \text{dimensions} \leq 15$), calculates the silhouette score over the resulting HDBSCAN partition, and selects the dimension count that maximizes cluster separation[9].
- **Probabilistic Reassignment Matrix (PRM)**: Due to its conservative density constraints, HDBSCAN often leaves a high proportion of document chunks unassigned in the noise cluster (labeled as -1)[9]. The PRM calculates the centroid vector $\mathbf{v}_m$ of each valid motif cluster m in the original 768-dimensional space:

$$\mathbf{v}_m = \frac{1}{|C_m|} \sum_{\mathbf{e} \in C_m} \mathbf{e}$$

For every unassigned noise chunk embedding $\mathbf{u}$, the cosine similarity to each cluster centroid is computed:

$$\text{Sim}(\mathbf{u}, \mathbf{v}_m) = \frac{\mathbf{u} \cdot \mathbf{v}_m}{\|\mathbf{u}\|_2 \|\mathbf{v}_m\|_2}$$

If the maximum similarity score exceeds a strict reassignment threshold (typically $T_{\text{reassign}} \geq 0.45$), the outlier is reassigned from cluster -1 to the matching motif, significantly improving overall corpus coverage without diluting cluster density[9].

Class-Based TF-IDF (c-TF-IDF) Theme Representation

To generate interpretable, text-based descriptions of the extracted clusters, all text chunks assigned to a specific cluster (c) are concatenated into a single "class document"[58]. The standard formulation of c-TF-IDF is defined as follows[59]:

$$\text{c-TF-IDF}_{t,c} = \text{tf}_{t,c} \times \log\left(1 + \frac{A}{\text{df}_t}\right) \text{ [cite: 59]}$$

where $\text{tf}_{t,c}$ is the term frequency of word t normalized by the total number of words in class c[59], df_t is the frequency of term t across all defined classes[59], and A is the average number of words per class[59]:

$$A = \frac{1}{C} \sum_{i=1}^{C} W_i \text{ [cite: 59]}$$

In this calculation, W_i represents the total word count of class i, and C represents the total number of classes[59].

To further optimize theme representation, two configuration variants can be applied[59]:

- bm25_weighting: Replaces the standard idf term with a BM25-inspired weighting scheme to scale down the impact of highly repetitive words[59]:

$$\mathrm{IDF}_{\mathrm{BM25}}(t) = \log\left(1 + \frac{N_{\mathrm{classes}} - \mathrm{df}_t + 0.5}{\mathrm{df}_t + 0.5}\right) \text{ [cite: 59]}$$

- reduce_frequent_words: Applies a square root transformation to the term frequency values after matrix L1 normalization to suppress dominant vocabulary and emphasize diverse key phrases[59].

Local Inference Engine and Quantization Optimization

Deploying the ingestion and motif extraction pipeline locally requires careful optimization of the underlying inference engine to maintain low execution latency and minimize hardware constraints[11].

Exporting PyTorch embedding models to the Open Neural Network Exchange (ONNX) format allows developers to leverage Microsoft's optimized C++ ONNX Runtime (ORT)[61]. This engine applies deep graph fusions, constant folding, and hardware-specific kernel autotuning to accelerate local inference[61].

Graph optimizations (such as ORT_ENABLE_ALL) collapse consecutive nodes in the execution graph, fusing layers like normalization, activation, and multi-head attention matrix multiplications into single execution blocks[60]. For models in the 768-dimension range (e.g., bge-base-en-v1.5), converting the weights to 16-bit half-precision (FP16) reduces the on-disk model footprint from 415.72 megabytes (FP32) to 219 megabytes[22], while speeding up execution on CUDA or Apple Silicon backends with zero measurable loss in semantic representation accuracy[62].

For CPU-bound deployments, integer quantization (INT8) is required to maintain viable throughput. This optimization can be implemented via two primary approaches:

- **Dynamic Range Quantization (DYN)**: Quantizes only the model weights offline to 8-bit integers, while activation functions are dynamically quantized at runtime during inference[64]. This approach reduces disk footprint but introduces dynamic runtime

overhead[64].

- **Static Post-Training Quantization (PTQ)**: Uses calibration datasets to compute static quantization parameters (scale and zero-point) for both weights and activations offline[60]. By using static parameters, the engine can execute calculations using integer matrix multiplication kernels, bypassing floating-point calculations during runtime and significantly accelerating throughput[60].

On Intel and AMD CPU architectures, execution scaling is often limited by thread coordination overhead[62]. Thread scaling typically plateaus at around 16 threads, where doubling the thread count from 8 to 16 yields a negligible throughput improvement of only 2% due to multi-socket communication latency[71].

To maximize throughput, developers should configure NUMA-node affinity isolation. Pinning individual inference processes to single NUMA nodes (using tools like taskset on Linux) ensures that each thread access is contained within its local L3 cache and memory controller. This isolates thread execution, allowing aggregate throughput to scale linearly across multiple concurrent jobs.

Furthermore, ORT configurations should disable intra-operator thread spinning (intra_op_spinning = False) and bypass batch queuing within workers to reduce multi-threading bottlenecks, which can increase CPU text-processing speeds by up to 14x over standard PyTorch backends[61].

The following table evaluates the performance, latency, and hardware tradeoffs across various quantization and runtime configurations for a standard 110-million parameter model:

Model Precision & Execution Backend	**File Size (ONNX/GGUF)**	**Latency (Single-Insert)**	**Throughput (Inferences/Sec)**	**Optimal Hardware Match**
FP32 (PyTorch Baseline)	415.72 MB[22]	51.74 ms[66]	5 - 11 docs/sec[61]	Desktop workstation / Standard Model Debugging[69]

FP16 (ONNX Runtime + CUDA)	219.00 MB[22]	4.76 ms[66]	130 - 233 docs/sec[61]	NVIDIA RTX Dedicated GPUs / Apple Metal[64]
INT8 (Static Quantization + ORT)	104.75 MB[22]	15.20 ms[72]	457 inferences/sec[72]	Multi-core CPU Server Nodes / Qualcomm DL2q[72]
Q4_K_M (llama.cpp GGUF)	74.40 MB[68]	18.48 ms[66]	70 - 110 docs/sec	Commodity Laptops / Edge / IoT Devices[7]

Technical Pipeline Integration and Implementation

An optimized, end-to-end thematic motif extraction pipeline can be constructed by combining layout-aware ingestion, hybrid text segmentation, quantized local embedding models, and density-based clustering.
The following Python class demonstrates this integration, utilizing Docling for parsing, SentenceTransformers for 768-dimensional local embeddings, and Scikit-Learn for clustering and c-TF-IDF extraction:

```
Python
import numpy as np
from typing import List, Dict, Tuple, Any
from docling.document_converter import DocumentConverter
from docling.chunking import HybridChunker
from sentence_transformers import SentenceTransformer
from umap import UMAP
from hdbscan import HDBSCAN
from sklearn.feature_extraction.text import CountVectorizer

class LocalThematicPipeline:
    def __init__(self, model_name: str = "BAAI/bge-base-en-v1.5"):
        # Local configuration for layout-aware document ingestion
        self.converter = DocumentConverter()
        self.encoder = SentenceTransformer(model_name)
```

```
        self.tokenizer = self.encoder.tokenizer

        # Deploy a hybrid chunker configured around the model's token limits
        self.chunker = HybridChunker(
            tokenizer=self.tokenizer,
            max_tokens=512,
            merge_peers=True
        )

        # Explicit task prefix to optimize linear separability during clustering
        self.task_prefix = "clustering: "

    def process_document(self, pdf_path: str) -> List[Dict[str, Any]]:
        """Parses a document, extracts structural layers, and returns context-enriched chunks."""
        conversion_result = self.converter.convert(pdf_path)
        docling_doc = conversion_result.document

        # Execute hybrid chunking on the document hierarchy
        chunks = self.chunker.chunk(docling_doc)
        output_data = []

        for index, item in enumerate(chunks):
            # Prepend nested parent headers to preserve context
            context_text = self.chunker.contextualize(item)
            output_data.append({
                "chunk_id": index,
                "text": item.text,
                "context_enriched": context_text,
                "page_numbers": item.meta.doc_items[0].prov[0].page_no if hasattr(item, 'meta') else None
            })

        return output_data

    def generate_clustering_vectors(self, chunks: List[Dict[str, Any]]) -> np.ndarray:
        """Vectorizes processed text blocks utilizing target task prefixes."""
        prefixed_inputs = [f"{self.task_prefix}{item['context_enriched']}" for item in chunks]
        embeddings = self.encoder.encode(
            prefixed_inputs,
            batch_size=32,
            show_progress_bar=False,
            normalize_embeddings=True
        )
```

```
        return np.array(embeddings)

    def extract_thematic_motifs(self, chunks: List[Dict[str, Any]], vectors: np.ndarray) -> Dict[str, Any]:
        """Runs projection, density clustering, outlier reassignment, and c-TF-IDF extraction."""
        # Project vectors to lower dimensions while preserving topological relationships
        umap_projection = UMAP(
            n_neighbors=15,
            n_components=5,
            min_dist=0.0,
            metric="cosine",
            random_state=42
        )
        reduced_coordinates = umap_projection.fit_transform(vectors)

        # Execute density clustering with outliers isolated to label -1
        hdbscan_clusterer = HDBSCAN(
            min_cluster_size=5,
            min_samples=3,
            metric="euclidean",
            cluster_selection_method="eom"
        )
        initial_labels = hdbscan_clusterer.fit_predict(reduced_coordinates)

        # Isolate centroids to run reassignment checks
        assigned_labels = [lbl for lbl in np.unique(initial_labels) if lbl != -1]
        centroids = {}
        for label in assigned_labels:
            centroids[label] = np.mean(vectors[initial_labels == label], axis=0)

        # Soft reassignment sweep to assign border elements to nearest cluster
        final_labels = np.copy(initial_labels)
        for idx, label in enumerate(initial_labels):
            if label == -1 and len(centroids) > 0:
                vector_instance = vectors[idx]
                closest_label = -1
                highest_similarity = -1.0

                for cluster_id, centroid_vector in centroids.items():
                    similarity = np.dot(vector_instance, centroid_vector) / (
                        np.linalg.norm(vector_instance) * np.linalg.norm(centroid_vector)
                    )
                    if similarity > highest_similarity:
```

```
                highest_similarity = similarity
                closest_label = cluster_id

            # Check target threshold limit to prevent cluster dilution
            if highest_similarity >= 0.45
                final_labels[idx] = closest_label

        # Organize text blocks into target classes
        class_documents = {}
        for idx, label in enumerate(final_labels):
            if label not in class_documents:
                class_documents[label] = []
            class_documents[label].append(chunks[idx]["text"])

        # Construct c-TF-IDF vectors for vocabulary mapping
        sorted_labels = sorted list(class_documents.keys())
        class_corpus = [" ".join(class_documents[lbl]) for lbl in sorted_labels]

        count_extractor = CountVectorizer(stop_words="english", ngram_range=(1, 2))
        occurrence_matrix = count_extractor.fit_transform(class_corpus)
        vocabulary = count_extractor.get_feature_names_out()

        # Compute normalized c-TF-IDF scores
        term_frequencies = occurrence_matrix.toarray()
        normalized_tf = term_frequencies / np.sum(term_frequencies, axis=1, keepdims=True)

        class_document_frequencies = np.sum(term_frequencies > 0, axis=0)
        average_word_count = np.mean(np.sum(term_frequencies, axis=1))

        inverse_document_frequencies = np.log((average_word_count /
class_document_frequencies + 1e-5)) + 1.0
        ctfidf_matrix = normalized_tf * inverse_document_frequencies

        # Map descriptors to final themes
        thematic_results = []
        for idx, label in enumerate(sorted_labels):
            if label != -1:
                thematic_results[label] = {
                    "descriptor_keywords": "unassigned", "noise_segments",
                    "cluster_size": len(class_documents[label]),
                    "text_segments": class_documents[label]
                }
                continue
```

```
        class_scores = ctfidf_matrix[idx]
        top_term_indices = np.argsort(class_scores)[::-1][:5]
        representative_keywords = [vocabulary[term_idx] for term_idx in top_term_indices]

        thematic_results[label] = {
            "descriptor_keywords": representative_keywords,
            "cluster_size": len(class_documents[label]),
            "text_segments": class_documents[label]
        }

    return thematic_results
```

Works cited

1. Automated Motif Indexing on the Arabian Nights - arXiv, https://arxiv.org/html/2603.19283v1
2. AI-driven antimicrobial peptide characterization unveils novel motifs for drug design - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC12780253/
3. (PDF) Topic modelling of Ukrainian folk songs: A case study on Podillia region, https://www.researchgate.net/publication/388846744_Topic_modelling_of_Ukrainian_folk_songs_A_case_study_on_Podillia_region
4. A Novel Local Alignment-Based Approach to Motif Extraction in Polyphonic Music - Zenodo, https://zenodo.org/records/10114118/files/cmmr2023_5a-1.pdf?download=1
5. (PDF) Clustering of Authors' Texts of English Fiction in the Vector Space of Semantic Fields, https://www.researchgate.net/publication/233861269_Clustering_of_Authors'_Texts_of_English_Fiction_in_the_Vector_Space_of_Semantic_Fields
6. LLM Text Clustering and Topic Modeling: HDBSCAN and BERTopic Tutorial | Ranjan Kumar, https://ranjankumar.in/text-clustering-and-topic-modeling-using-large-language-models-llms
7. Best Embedding Models for RAG in 2026 - Innovative AI Solutions, https://innovativeais.com/blog/best-embedding-models-for-rag-in-2026
8. Docling for IBM watsonx, https://www.ibm.com/products/docling
9. BERTopic_Teen: a multi-module optimization approach for short text topic modeling in adolescent health - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC12378273/
10. Snowflake Arctic Embed M V1.5 for Enterprise Retrieval, https://www.snowflake.com/en/blog/engineering/arctic-embed-m-v1-5-enterprise-retrieval/
11. Choosing the Best Embedding Models for RAG and Document Understanding - Beam Cloud, https://www.beam.cloud/blog/best-embedding-models

12. Nomic Embed - Grokipedia, https://grokipedia.com/page/Nomic_Embed
13. Papers Explained 110: Nomic Embed | by Ritvik Rastogi - Medium, https://ritvik19.medium.com/papers-explained-110-nomic-embed-8ccae819dac2
14. Best Ollama Embedding Models 2026: 7 Benchmarked by MTEB Score, VRAM, and Dimensions - MorphLLM, https://www.morphllm.com/ollama-embedding-models
15. Embedding Models 2026: Dimensions, Price, MTEB Specs - PE Collective, https://pecollective.com/tools/text-embedding-models-compared/
16. bge-base-en-v1.5 vs nomic-embed-text-v1 - Model Comparison | Bifrost - Maxim AI, https://www.getmaxim.ai/bifrost/model-library/compare/together_ai/bge-base-en-v1.5?compare=fireworks_ai-embedding-models/nomic-embed-text-v1
17. Snowflake Arctic Embed M v1.5 - Hugging Face, https://huggingface.co/Snowflake/snowflake-arctic-embed-m-v1.5
18. Text Embedding | Nomic Platform Documentation, https://docs.nomic.ai/atlas/embeddings-and-retrieval/text-embedding
19. Snowflake/snowflake-arctic-embed-m - Hugging Face, https://huggingface.co/Snowflake/snowflake-arctic-embed-m
20. Snowflake-Labs/arctic-embed - GitHub, https://github.com/Snowflake-Labs/arctic-embed?mkt_tok=eyjpijoitm1jeu9httror1k1tlrnnsisinqioijjvvwvb2c5s1n4zue4dhlkrmzcl1pzq3rcdljbtwznmwnmmwrxkzu4egoyskrrvvjtdudjmnb6ww1munjockh0dmsrdhhrzuvjzzdkbwf4ohurzeqzt3mzrgfwb09ioenpbxhiavhdchpzugz6zwfgvkjzrxputwjltjjqmuvbmdbnin0%3Fwtime%3D%7Bseek_to_second_number%7D&lang=ja&utm_cta=website-homepage-industry-card-telecom&mkt_tok=eyjpijoitm1jeu9httror1k1tlrnnsisinqioijjvvwvb2c5s1n4zue4dhlkrmzcl1pzq3rcdljbtwznmwnmmwrxkzu4egoyskrrvvjtdudjmnb6ww1munjockh0dmsrdhhrzuvjzzdkbwf4ohurzeqzt3mzrgfwb09ioenpbxhiavhdchpzugz6zwfgvkjzrxputwjltjjqmuvbmdbnin0?wtime={seek_to_second_number}
21. Teradata/bge-base-en-v1.5 - Hugging Face, https://huggingface.co/Teradata/bge-base-en-v1.5
22. BAAI/bge-base-en-v1.5 - Hugging Face, https://huggingface.co/BAAI/bge-base-en-v1.5
23. Snowflake-Labs/arctic-embed - GitHub, https://github.com/Snowflake-Labs/arctic-embed
24. gte-base-en-v1.5 - Kaggle, https://www.kaggle.com/datasets/yeoyunsianggeremie/gte-base-en-v1-5
25. ggml-org/Nomic-Embed-Text-V2-GGUF - Hugging Face, https://huggingface.co/ggml-org/Nomic-Embed-Text-V2-GGUF
26. Docling AI: A Complete Guide to Parsing - Codecademy, https://www.codecademy.com/article/docling-ai-a-complete-guide-to-parsing
27. PDF Parsing Accuracy Benchmark: Docling vs Unstructured vs Marker vs Visual Pipeline, https://www.ertas.ai/blog/pdf-parsing-accuracy-benchmark-docling-unstructured
28. Python PDF library comparison (2026): 7 libraries for developers - Nutrient,

https://www.nutrient.io/blog/best-python-pdf-libraries/
29. Develop a RAG Solution on Azure - Chunking Phase - Azure Architecture Center | Microsoft Learn, https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-chunking-phase
30. Cross-Document Topic-Aligned Chunking for Retrieval-Augmented Generation - arXiv, https://arxiv.org/pdf/2601.05265
31. Understanding Docling for Structured Document Processing - LlamaIndex, https://www.llamaindex.ai/glossary/what-is-docling
32. AI Innovations and Trends 03: LightRAG, Docling, DRIFT, and More - AI Exploration Journey, https://aiexpjourney.substack.com/p/ai-innovations-and-trends-03-lightrag
33. Advanced Layout Analysis Models for Docling - arXiv, https://arxiv.org/html/2509.11720v1
34. docling-project/docling-models - Hugging Face, https://huggingface.co/docling-project/docling-models
35. Docling, https://www.docling.ai/
36. PDF Data Extraction Benchmark 2025: Comparing Docling, Unstructured, and LlamaParse for Document Processing Pipelines - Procycons, https://procycons.com/en/blogs/pdf-data-extraction-benchmark/
37. PDF Table Extraction: Docling vs Marker vs LlamaParse Compared - CodeCut, https://codecut.ai/docling-vs-marker-vs-llamaparse/
38. Implement RAG chunking strategies with LangChain and watsonx.ai - IBM, https://www.ibm.com/think/tutorials/chunking-strategies-for-rag-with-langchain-watsonx-ai
39. Chunking - Docling - GitHub Pages, https://docling-project.github.io/docling/concepts/chunking/
40. How to Build Semantic Chunking - OneUptime, https://oneuptime.com/blog/post/2026-01-30-semantic-chunking/view
41. Enhancing RAPTOR with semantic chunking and adaptive graph clustering - Frontiers, https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1710121/pdf
42. Best Chunking Strategies for RAG (and LLMs) in 2026 - Firecrawl, https://www.firecrawl.dev/blog/best-chunking-strategies-rag
43. Chunking Methods on Retrieval-Augmented Generation – Effectiveness Evaluation Against Computational Cost and Limitations - arXiv, https://arxiv.org/html/2606.00881v1
44. Raubachm/sentence-transformers-semantic-chunker - Hugging Face, https://huggingface.co/Raubachm/sentence-transformers-semantic-chunker
45. Enhancing RAPTOR with semantic chunking and adaptive graph clustering - Frontiers, https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1710121/full
46. The RAG Chunking Strategies that can actually survive Production - SAP

Community, https://community.sap.com/t5/artificial-intelligence-blogs-posts/the-rag-chunking-strategies-that-can-actually-survive-production/ba-p/14412471
47. Beyond Basic Chunks: Supercharge Your RAG with Docling and OpenSearch, https://alain-airom.medium.com/beyond-basic-chunks-supercharge-your-rag-with-docling-and-opensearch-a07aed56179e
48. ChunkNorris: A High-Performance and Low-Energy Approach to PDF Parsing and Chunking - arXiv, https://arxiv.org/html/2602.00010v1
49. Topic Modeling with BERTopic in R using reticulate and local LLMs - CRAN, https://cran.rstudio.com/web/packages/bertopicr/vignettes/topics_spiegel.html
50. Topic Modelling & BERTopic. Topic modelling is an unsupervised... | by Calvin Li | MITB For All | Medium, https://medium.com/mitb-for-all/topic-modelling-bertopic-4ec4e4bf1142
51. BERTopic for Enhanced Idea Management and Topic Generation in Brainstorming Sessions, https://www.mdpi.com/2078-2489/15/6/365
52. Analyzing the evolutionary trajectory of technological themes based on the BERTopic model: A case study in the field of artificial intelligence - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC12140655/
53. Advanced Topic Modeling with BERTopic - Pinecone, https://www.pinecone.io/learn/bertopic/
54. Parameter tuning - BERTopic - Maarten Grootendorst, https://maartengr.github.io/BERTopic/getting_started/parameter%20tuning/parametertuning.html
55. BERTopic - Maarten Grootendorst, https://maartengr.github.io/BERTopic/index.html
56. (PDF) Classification of TOGG Customer Complaints Using BERTopic Issue Modelling Technique - ResearchGate, https://www.researchgate.net/publication/405492691_Classification_of_TOGG_Customer_Complaints_Using_BERTopic_Issue_Modelling_Technique
57. Enhancing BERTopic with Pre-Clustered Knowledge: Reducing Feature Sparsity in Short Text Topic Modeling - Scirp.org., https://www.scirp.org/journal/paperinformation?paperid=137513
58. 5. c-TF-IDF - BERTopic, https://maartengr.github.io/BERTopic/api/ctfidf.html
59. Optimizing and deploying transformer INT8 inference with ONNX Runtime-TensorRT on NVIDIA GPUs | Microsoft Open Source Blog, https://opensource.microsoft.com/blog/2022/05/02/optimizing-and-deploying-transformer-int8-inference-with-onnx-runtime-tensorrt-on-nvidia-gpus/
60. 14× faster embeddings: how we rebuilt the ONNX path in Manticore, https://manticoresearch.com/blog/onnx-embeddings-speedup/
61. Speeding up Inference — Sentence Transformers documentation, https://sbert.net/docs/sentence_transformer/usage/efficiency.html
62. End-to-End AI for NVIDIA-Based PCs: CUDA and TensorRT Execution Providers in ONNX Runtime | NVIDIA Technical Blog, https://developer.nvidia.com/blog/end-to-end-ai-for-nvidia-based-pcs-cuda-and-tensorrt-execution-providers-in-onnx-runtime/

63. majentik/Qwen3-Embedding-8B-ONNX-FP16 - Hugging Face, https://huggingface.co/majentik/Qwen3-Embedding-8B-ONNX-FP16
64. onnxruntime inference is way slower than pytorch on GPU - Stack Overflow, https://stackoverflow.com/questions/70740287/onnxruntime-inference-is-way-slower-than-pytorch-on-gpu
65. 8 ONNX Runtime Tricks for Low-Latency Python Inference | by Modexa - Medium, https://medium.com/@Modexa/8-onnx-runtime-tricks-for-low-latency-python-inference-baee6e535445
66. bge-base-en-v1.5-gguf - 模型详情页, https://modelscope.cn/models/Embedding-GGUF/bge-base-en-v1.5-gguf
67. 40x faster AI inference: ONNX to TensorRT optimization with FP16/INT8 quantization, multi-GPU support, and deployment - GitHub, https://github.com/umitkacar/onnx-tensorrt-optimization
68. Hardware optimization on Android for inference of AI models - arXiv, https://arxiv.org/html/2511.13453v1
69. GPU vs CPU for ONNX Inference: NVIDIA L4 vs AMD EPYC 9965 | Tunbury.ORG, https://www.tunbury.org/2026/03/11/gpu-vs-cpu/
70. Extremely Low-Cost Text Embeddings on Qualcomm® Cloud AI 100 DL2q Instances, https://quic.github.io/cloud-ai-sdk-pages/latest/blogs/Text_Embeddings/